



THU
Technische
Hochschule
Ulm

HENSOLDT

Bachelor thesis at Ulm University of Applied Sciences

Department of Computer Science

Degree Program Informatik

Presented by Gabriel Zimmermann

April 2026

1. Reviewer: Prof. Dr.-Ing. Klaus Baer
2. Reviewer: Prof. Dr.-Ing. Philipp Graf

Place, Date, Signature

Diese Abschlussarbeit wurde von mir selbständig verfasst. Es wurden nur die angegebenen Quellen und Hilfsmittel verwendet. Alle wörtlichen und sinngemäßen Zitate sind in dieser Arbeit als solche kenntlich gemacht.

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references. Place, date and signature.

Place, Date, Signature

Plugin System's in Rust

Gabriel Zimmermann

I. Abstract

Plugin architectures enable extensible software systems by allowing runtime loading of external modules. While Rust offers memory safety guarantees and performance characteristics ideal for plugin development, the language's lack of a stable Application Binary Interface (ABI) presents significant challenges for cross-module compatibility. This thesis examines the landscape of plugin system implementations in Rust, evaluating three primary approaches: native Rust plugins, C ABI-compatible interfaces, and Inter-Process Communication (IPC) mechanisms. Through comparative analysis of technical requirements, performance implications, and maintenance considerations, this work demonstrates that Rust's unstable ABI hampers reliable direct plugin loading across compiler versions. The findings indicate that C ABI remains the optimal choice when performance is paramount, while IPC-based architectures provide superior isolation and version compatibility at the cost of increased overhead. These results contribute practical guidance for developers designing extensible Rust applications and highlight ongoing gaps in Rust's ecosystem for safe, performant plugin architectures.

Contents

I)	Abstract	3
II)	Introduction	7
III)	Related work	8
III.A)	Existing work	8
III.B)	Foundational Concepts	8
III.B.a)	Application Binary Interface (ABI)	8
III.B.b)	Rust Types and ABI Instability	9
III.B.c)	Plugin Systems	9
III.B.d)	Dynamic Linking and Runtime Loading	9
IV)	Methods	11
IV.A)	Analysis of the requirements	11
IV.A.a)	Performance Overhead	11
IV.A.b)	Development complexities	11
IV.A.c)	Limitations	12
IV.A.d)	Safety	12
IV.A.e)	Interoperability	12
IV.B)	Potential approaches and problems	12
IV.B.a)	Unstable Rust ABI	13
IV.B.b)	abi_stable crate	13
IV.B.c)	rust-bindgen	13
IV.B.d)	cbindgen crate	13
IV.B.e)	stabby crate	13
IV.B.f)	Other Rust Bindings	13
IV.B.f.a)	pyo3 (Python bindings)	13
IV.B.f.b)	WebAssembly (Wasm)	14
IV.B.g)	Interprocess Communication (IPC)	14
IV.B.g.a)	rust_bridge crate	14
IV.B.g.b)	other ipc crates? (overview)	14
IV.C)	Selected approach and detailed solutions	14
V)	Results	15
V.A)	Validation of the overall concept	15
V.B)	Description and motivation for the test cases	15
V.C)	Overview and evaluation of the acquired results	15
V.C.a)	Unstable Rust ABI	15
V.C.b)	Stable Rust ABI (crate)	15
V.C.c)	stabby (crate)	15
V.C.d)	rust_bridge (crate)	15
VI)	Conclusion and Future Work	16
VII)	Listings	17
	References	18

Abbreviations:

- ABI: Application Binary Interface
- IPC: Interprocess Communication
- RPC: Remote Procedure Call
- ZST: Zero-Sized Type

II. Introduction

Modern software systems increasingly demand flexibility and extensibility. Plugin architectures have emerged as a fundamental pattern for achieving this goal, enabling applications to load additional functionality at runtime without recompilation. As highlighted by Apple's technical documentation, "Plug-in architectures are an attractive solution for developers seeking to build applications that are modular, customizable, and easily extensible" [1].

The rise of the rust language popularity has led to an increased interest in using Rust for plugin development. Rust has established itself as a powerful systems programming language that offers memory safety guarantees and performance characteristics. However, the language's design philosophy prioritizes compile-time guarantees over binary stability, resulting in an intentionally unstable Application Binary Interface (ABI). Small changes in your written code, can impact the resulting binary in a way that it is not compatible with the previous version. With that being said, we can circumvent this problem by using other approaches.

One popular approach is to use C ABI-compatible interfaces, which can provide a stable interface for plugins while still allowing developers to write their plugins in Rust. The C ABI has been the backbone for many plugin systems in various programming languages, including the Rust ecosystem. But this approach requires the careful handling of unsafe code and can lead to some performance overhead due to the need for FFI (Foreign Function Interface) calls. In addition, this can lead to the loss of Rust's safety guarantees, as the C ABI does not provide the same level of safety as Rust's native interfaces.

Another approach is to use Inter-Process Communication (IPC) mechanisms, which can provide a more flexible and scalable architecture for plugin systems. However, this approach can also introduce additional complexity and overhead, as it requires the management of separate processes and communication channels. This will impact the performance of the plugin system, as IPC can introduce latency and reduce the overall efficiency of the system. However, it can provide better isolation and security for plugins, as they run in separate processes.

In this thesis, we are interested in understanding the advantages and disadvantages of the stated approaches and what Crates are offered within the Rust Ecosystem. Later on, we will evaluate the different approaches based on the performance overhead using a simple benchmark, the resulting development complexity, limitations and interoperability.

III. Related work

A. Existing work

As of 2026, the Rust ecosystem remains in a state of deliberate ABI instability, a condition that has persisted despite years of community advocacy and formal proposals. The most significant attempt to address this were RFC 1675 [2] and RFC 600 [3] (“Make Rust ABI stable enough to provide plugins functionality”), which proposed a “stable modular ABI” to enable runtime loading of Rust libraries without C interop. However, the proposal was ultimately deferred, with the compiler team citing the high cost of freezing internal representations that are critical for future optimizations, such as constant evaluation and monomorphization strategies [2]. Additionally, it was proposed to allow for a modular ABI, where a developer can specify the ABI by using macro system. [4]. Recent discourse at conferences like RustConf and FOSDEM reinforces this stance; speakers such as Niko Matsakis and others have emphasized that stabilizing the ABI would “lock in” implementation details, potentially hindering the compiler’s ability to evolve (Matsakis, RustConf 2023). Instead of a native solution, the community has converged on pragmatic workarounds. The prevailing consensus, reflected in technical talks and blog posts from 2023-2024, is that the C ABI remains the only reliable standard for binary interoperability, forcing developers to adopt verbose extern “C” interfaces or rely on third-party crates like `abi_stable` to simulate stability through `repr(C)` wrappers and runtime checks (NullDeref, 2023; Arroyo Blog, 2023). While experimental efforts continue to explore WebAssembly as a language-agnostic ABI alternative, no native Rust-to-Rust stable ABI has been implemented, leaving plugin developers to navigate a fragmented landscape of manual FFI management and abstraction layers.

TODO: Fix links with the concrete answer of the dev team...

Currently only rust std library and ferrocene compiler verified. Therefore, most code that needs to be verified is already limited and the solutions that can be used for Plugins are limited. Not the focus of this thesis.

B. Foundational Concepts

To contextualize the challenges of plugin systems in Rust, it is essential to establish precise definitions of the core architectural components involved: the ABI, how stable the Rust Type System is for the ABI, plugin architectures, and dynamic linking mechanisms.

a) Application Binary Interface (ABI):

While the Application Programming Interface (API) defines the source-code level contract between software components (function names, argument types, and headers), the Application Binary Interface (ABI) operates at a lower level, governing the interaction between compiled machine code.

Definition: According to the System V Application Binary Interface, the standard for Unix-like operating systems, the ABI is “an interface between two binary program modules” (System V ABI, 2023). It encompasses:

- **Calling Conventions:** How function arguments are passed (registers vs. stack), how return values are delivered, and who is responsible for cleaning up the stack (caller vs. callee).
- **Data Layout:** The memory alignment, padding, and byte-ordering (endianness) of data structures.
- **Name Mangling:** The algorithm used by compilers to encode function names (including type information) into symbol names for the linker.
- **Exception Handling:** The mechanism for propagating errors or exceptions across module boundaries.

Relevance to Rust: In the context of Rust, the lack of a standardized ABI is a deliberate design choice. As noted by the Rust compiler team, the ABI is “deliberately unstable” to allow the compiler to optimize memory layouts and calling conventions aggressively without being constrained by backward compatibility requirements (Rust Internals, 2020).

This contrasts with the C ABI, which is strictly defined and stable, serving as the de facto standard for binary interoperability.

b) *Rust Types and ABI Instability:*

[5]

Product types (or structs): is it ordered? is it aligned?

C is always aligned and source-ordered.

Rust fields also aligned, but the order is minimized for memory usage.

Rust can enforce this by using `repr(C)`, but this is not the default, and the compiler can change the layout for optimization purposes. This means that without explicit annotations, the memory layout of a Rust struct is not guaranteed to be stable across different compiler versions or optimization levels, which can lead to binary incompatibility when used in a plugin system.

Sum types (or enums): how to distinguish the variants? where is the data?

Not supported by C. Can be done using a tagged union, but you need to give each type a specific number (usually an enum).

Rust may be put in a niche (like common padding or using type's forbidden value).

Example with `Cow<'a, str>`: Can waste 7 bytes for padding on a 64-bit architecture.

Rust guarantees that the layout of a sum type is not clashing with the layout of a product type.

Unit types (`()`): are they zero-sized? what about references?

Doesn't exist in C as zero-sized type. ZSTs are undefined behavior.

As Pierre states, for a stable ABI all linkers must agree on an ABI for each symbol [5] (double check, maybe I need to link the talk instead...).

Note stable due to changes in compiler version, optimization level, target architecture, using `-Z randomize-layout` and so on. This is a fundamental problem for plugin systems in Rust, as the host and the plugin may be compiled with different versions of the compiler or different optimization flags, leading to binary incompatibility.

A few examples as to why the Rust ABI is not stable yet can be found in niche optimizations: enums with multiple data variant optimizations (1.65), field ordering optimizations (1.67), `Cow<str>` regression (1.70), and more.

c) *Plugin Systems:*

A plugin system is a software architecture pattern that allows a host application to be extended by third-party modules (plugins) without modifying the host's source code or recompiling the entire system.

Definition: In software engineering literature, a plugin is defined as "a software component that adds a specific feature to an existing computer program" (Gamma et al., 1994). Key characteristics include:

- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.

Architectural Patterns: Plugin systems generally follow one of two patterns:

- **Static Linking:** Plugins are compiled into the host binary. This offers performance benefits but lacks runtime flexibility.
- **Dynamic Linking:** Plugins are separate binary files loaded into the host process memory space at runtime. This is the primary focus of this thesis, as it necessitates strict ABI compatibility or robust isolation mechanisms.

The primary challenge in Rust plugin systems is ensuring that the host and the plugin, potentially compiled with different compiler versions or optimization flags, agree on the binary representation of data and function calls.

d) *Dynamic Linking and Runtime Loading:*

Dynamic Linking is the process of resolving references to external libraries or modules at runtime, rather than at compile time (link time).

Mechanism: In Unix-like systems, this is achieved through the `dlopen` (dynamic open), `dlsym` (dynamic symbol lookup), and `dlclose` (dynamic close) system calls. In Windows, the equivalents are `LoadLibrary`, `GetProcAddress`, and `FreeLibrary`.

Process Flow:

- **Discovery:** The host application locates the plugin binary file (e.g., `.so`, `.dll`, `.dylib`).
- **Loading:** The operating system maps the plugin's code and data segments into the host's virtual memory space.
- **Resolution:** The host resolves symbol addresses (function pointers) within the loaded plugin.
- **Execution:** The host invokes the plugin's functions via the resolved pointers.

Challenges in Rust: Dynamic linking in Rust is complicated by the language's ownership model and panic handling. Since Rust does not guarantee a stable ABI, a plugin compiled with Rust version 1.70 might use a different struct layout or calling convention than a host compiled with version 1.75. Furthermore, if a plugin panics, the unwinding behavior must be contained; otherwise, it can lead to undefined behavior or crashes in the host process. As described in implementation guides, "panics cannot cross the FFI boundary" without explicit handling, such as wrapping function bodies in `catch_unwind` or using ABI-stable libraries like `abi_stable` [6].

No comprehensive comparison between possible solutions for plugin systems in Rust has been done yet. There are some simple comparisons, but they don't cover the full range of options and trade-offs.

IV. Methods

A. Analysis of the requirements

Goal of the research was to analyze the following points:

- Performance Overhead IV.A.a
- Development Complexities IV.A.b
- Limitations IV.A.c
- Safety IV.A.d
- Interoperability IV.A.e

a) Performance Overhead:

When measuring our plugins, we expect that the execution time of the plugin's internal functionality to be relatively similar across different approaches. However, the setup time for each approach is expected to vary, with approaches that require more complex initialization (e.g. loading a dynamic library, setting up IPC communication, serialization) likely to require more time.

We expect that the IPC approach will have higher performance overhead compared to the other approaches due to the need for serialization and deserialization of data, as well as the communication overhead between processes. This is because IPC typically involves more complex interactions between processes, which can introduce additional latency compared to approaches that operate within a single process. We expect the measurements for the Rust ABI and C ABI approaches to be more similar to each other, as they both involve loading and executing code within the same process, albeit with different calling conventions and potential overhead from FFI in the case of the C ABI approach.

To measure the results, we are using the *criterion* and *gungrau* crate for our benchmarks. The results of these will be separated into setup time and function execution time, which allows us to analyze the overhead of each approach in more detail.

As noted by the *criterion* documentation, the *criterion* crate is a statics-driven micro-benchmarking library, which aims to provide strong statistical confidence in detecting and estimating the size of performance improvements and regressions [7]. As stated by the *gungrau* documentation, "gungrau is a one-shot benchmarking harness and framework which uses Valgrind's Callgrind, Cachegrind, and DHAT to provide extremely accurate and consistent measurements of Rust code, making it perfectly suited to run in environments like a CI" [8].

We hope by combining both of these tools, we can get a comprehensive understanding of the performance characteristics of each approach, and how they compare to each other in terms of setup time and function execution time.

In Rust benchmarking, wrapping inputs and intermediate values in `black_box` is essential to prevent the compiler from optimizing away the code you intend to measure. Because Rust's optimizer is aggressive, it may detect that a function's result is unused or that an input is constant, leading it to eliminate the entire computation or hoist it out of the loop, resulting in artificially low (and inaccurate) timing data. The `std::hint::black_box` function acts as an opaque barrier, signaling to the compiler that the value is unknown and must be treated as having side effects, thereby forcing it to execute the code exactly as written within the benchmark. [9] Each benchmark will be run with and without `black_box` to compare the results and understand the impact of compiler optimizations on our measurements.

b) Development complexities:

When using external crates, they usually come with their own learning curve and documentation. This can add to the development time, especially if the crate is not well-documented or has a complex API. Additionally, integrating external crates into an existing codebase can sometimes lead to compatibility issues or require significant refactoring. Quantifying this is difficult, as it depends on the developer's experience, familiarity with the crate, and the specific requirements of the project. We have decided to instead focus

on a qualitative analysis of the development complexities, based on the given available data by the crates maintainers and our experience during the implementation of each approach. This will include factors such as the ease of use of the crate's API, the quality of its documentation, and any challenges faced during integration. After summarizing these, we will

We expect that the dynamic library loading approaches (Rust ABI and C ABI) may have higher development complexities compared to the IPC approach, due to the need to manage dynamic libraries, handle FFI (in the case of C ABI), and ensure compatibility between the plugin and host application. The IPC approach may have lower development complexities in terms of integration, as it allows for more decoupled communication between processes, but it may require more complex software design to handle serialization and inter-process communication effectively.

c) Limitations:

Each approach may come with some limitations for the developers. For example, some Rust Types cannot be easily represented in C, which can lead to limitations in the C ABI approach. We also expect that the IPC approach may have limitations in terms of the types of data that can be easily serialized and deserialized, which could impact the flexibility of this approach. Additionally, the IPC approach may require more complex software design, which could be a limitation for developers who are not familiar with this type of architecture.

Because we are also working with different ABI solutions, there can be complexities in regards of their restrictions and limitations.

In some cases, Limitations may overlap with Section IV.A.b, as certain limitations may require additional development effort to work around or mitigate. For example, if a particular approach has limitations in terms of the types of data that can be easily represented or serialized, this may require additional development effort to implement custom serialization logic or to design the plugin architecture in a way that avoids these limitations.

d) Safety:

The Rust Language is designed with safety in mind, and it provides strong guarantees against common programming errors such as null pointer dereferences, buffer overflows, and data races.

Yet, this does not result in formal verifications, like DO-178C or IEC 61508, which are often required in safety-critical domains. However, the Rust ecosystem has been making strides in this area, with efforts to achieve certifications for certain subsets of the Rust language and libraries, such as the recent achievement of IEC 61508 (SIL 2) certification for a subset of the Rust core library by Ferrous Systems [10], [11].

From this, we can infer that while Rust provides strong safety guarantees, the level of safety can depend on the specific approach used for plugin integration. For example, approaches that involve unsafe code (such as FFI in the C ABI approach) may require more careful handling to ensure safety, while approaches that operate entirely within safe Rust (such as the Rust ABI approach) may provide stronger safety guarantees by default. The IPC approach may also have its own safety considerations, such as ensuring that data is properly validated and that communication channels are secure.

For this thesis, we do not consider it appropriate to perform a quantitative or qualitative analysis of the safety of each approach, as this would require a detailed examination of the specific implementation and usage patterns, which may be beyond the scope of this research.

e) Interoperability:

How flexible are the presented approaches? Can they be linked with other libraries?

B. Potential approaches and problems

When developing a plugin system in Rust, there are several potential approaches that can be taken, each with its own set of advantages and disadvantages. In this section we will introduce different crates and give an overview of their functionality. We will have a closer look at the chosen approach in the next section Section IV.C.

a) *Unstable Rust ABI:*

It is possible to load other Rust Libraries as plugins, using the Rust ABI. However, this approach is complicated by the fact that the Rust ABI is not stable.

b) *abi_stable crate:*

The `abi_stable` crate enables stable, version-independent interoperability between Rust libraries and executables by generating a custom, C-like ABI that bypasses Rust's unstable internal layout guarantees, allowing dynamically linked shared objects to communicate safely even when compiled with different Rust compiler versions or feature flags. It achieves this through a macro-heavy approach (`#[sabi]`) that enforces strict trait bounds and generates wrapper code to handle memory layout, vtables, and function pointers, effectively creating a "Rust FFI" that is safer than raw extern "C" for complex types. However, this stability comes with significant trade-offs: the generated code introduces non-trivial runtime overhead due to extra indirection and dynamic dispatch, the API is verbose and requires extensive boilerplate that can obscure the underlying logic, and it currently lacks support for many modern Rust features like `async/await` or generic specialization, making it less suitable for performance-critical paths or projects prioritizing minimal binary size compared to simpler extern "C" or `cxx` solutions.

c) *rust-bindgen:*

The `bindgen` crate automates the creation of Rust FFI bindings by parsing C or C++ header files using Clang to generate Rust extern blocks and struct definitions that mirror the original C API, effectively eliminating the manual, error-prone process of translating C types into Rust equivalents. It operates by invoking Clang to analyze the header syntax tree, resolving macros and complex type definitions, and outputting a Rust source file that can be integrated into a build script (`build.rs`) to ensure bindings stay synchronized with header changes. However, this automation introduces significant downsides: the generated code is often verbose, unreadable, and difficult to maintain manually, leading to large binary sizes; it tightly couples the Rust build process to the presence of a compatible Clang installation and the exact C headers, complicating cross-compilation and CI pipelines; and because it exposes the raw C API directly, it frequently forces developers to wrap the generated unsafe code in safe abstractions to prevent memory safety issues, adding an extra layer of development overhead despite the initial automation.

d) *cbindgen crate:*

The `cbindgen` crate facilitates Rust-to-C interoperability by scanning Rust source code annotated with specific attributes (like `#[no_mangle]` and `repr(C)`) to automatically generate C header files (`.h`) and corresponding documentation, allowing C or C++ projects to consume Rust libraries as if they were native C code without needing to parse Rust syntax directly. It works by traversing the Rust AST to extract public functions, structs, and enums, translating Rust types into their C equivalents (e.g., mapping `u32` to `uint32_t` and `Option<T>` to a union or pointer pattern) and handling complex scenarios like callbacks and string lifetimes through configurable build scripts. However, its effectiveness is limited by the requirement that Rust code must strictly adhere to C-compatible layouts and calling conventions, often necessitating manual refactoring of idiomatic Rust code to fit these constraints; additionally, it struggles with advanced Rust features like generics, traits, and complex lifetimes which have no direct C equivalent, forcing developers to write verbose wrapper functions or accept that the generated headers may be incomplete or require significant manual patching to be usable in a C environment.

e) *stabby crate:*

f) *Other Rust Bindings:*

pyo3 (Python bindings):

The `pyo3` crate enables seamless bidirectional interoperability between Rust and Python by leveraging the Python C API to expose Rust functions, structs, and enums as native Python objects while allowing Rust code to call Python functions and manipulate Python objects with type safety. It operates through a macro-driven system where the `#[pyclass]` and `#[pymethods]` attributes generate the necessary C bindings and

glue code, automatically handling reference counting, memory management, and the conversion of Rust types (like `Vec` or `String`) into their Python equivalents (like `list` or `str`) via the `IntoPy` and `FromPyObject` traits. By utilizing the Global Interpreter Lock (GIL) guard to ensure thread safety during Python interactions and providing a high-level API that abstracts away the complexities of the underlying C API, `pyo3` allows developers to write performance-critical Python extensions in Rust that feel and behave like idiomatic Python code, all while maintaining the safety guarantees of the Rust compiler.

WebAssembly (Wasm):

Using WebAssembly (Wasm) with Rust involves compiling Rust code to the `wasm32-unknown-unknown` target, a process streamlined by tools like `wasm-pack` and `wasm-bindgen` that bridge the gap between Rust's memory model and JavaScript's environment. By annotating Rust functions with `wasm_bindgen`, developers can automatically generate the necessary glue code to export Rust functions to JavaScript and import JS functions into Rust, handling complex type conversions (such as mapping Rust Strings to JS strings and Vectors to typed arrays) and managing memory ownership across the boundary. This workflow typically starts by installing the `wasm32` target via `rustup`, writing Rust logic with the `wasm-bindgen` crate, running `wasm-pack build` to generate a `.wasm` binary and corresponding JavaScript/TypeScript bindings, and finally loading the resulting module in a browser or Node.js environment to execute high-performance, sandboxed code that interacts seamlessly with the host application.

g) Interprocess Communication (IPC):

different structure. communication over serialization

positive: interoperability, less complexity during development

negative: performance overhead, more complex software design

rust_bridge crate:

other ipc crates? (overview):

C. Selected approach and detailed solutions

When choosing the approach for further analysis, we decided to take a closer look at the Unstable Rust ABI, the `abi_stable` crate, the `stabby` crate and on the `rust_bridge` crate. We choose those, because they represent a good variety of approaches to plugin systems in Rust, and they are also the most relevant to our research goals. The Unstable Rust ABI approach is interesting because it represents the most direct way to load Rust plugins, but it also comes with significant challenges due to the instability of the Rust ABI. The `abi_stable` crate is interesting because it provides a solution to the instability of the Rust ABI, but it also comes with its own set of trade-offs in terms of performance and complexity. The `stabby` crate is interesting because it has a more fundamental approach (NOT HAPPY WITH THIS PHRASING!!) to plugin systems in Rust, and it also has a focus on safety and ease of use. Finally, the `rust_bridge` crate is interesting because it represents an IPC-based approach to plugin systems in Rust, which has its own set of advantages and disadvantages compared to the other approaches.

WebAssembly (Wasm) is not covered in this paper.

C ABI: `abi_stable` and `stabby`

Rust: ... well unstable

IPC: `rust_bridge`

V. Results

A. *Validation of the overall concept*

Validating the Performance Overhead is done using gungraun and criterion benchmarks and then comparing the results with each other.

B. *Description and motivation for the test cases*

C. *Overview and evaluation of the acquired results*

a) *Unstable Rust ABI:*

b) *Stable Rust ABI (crate):*

c) *stabby (crate):*

d) *rust_bridge (crate):*

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

VI. Conclusion and Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

VII. Listings

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

References

- [1] A. Inc., "Plug-in Architectures." Accessed: Apr. 01, 2026. [Online]. Available: <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/LoadingCode/Concepts/Plugins.html>
- [2] iddm, "Make Rust ABI stable enough to provide plugins functionality #1675." Accessed: Apr. 01, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/issues/1675>
- [3] steveklabnik, "Define a Rust ABI #600." Accessed: Apr. 01, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/issues/600>
- [4] isaac, "A Stable Modular ABI for Rust." Accessed: Apr. 07, 2026. [Online]. Available: <https://internals.rust-lang.org/t/a-stable-modular-abi-for-rust/12347>
- [5] T. S. Developers, "Stabby: Defining ABI-stable types." Accessed: Apr. 07, 2026. [Online]. Available: https://docs.rs/stabby/latest/stabby/_tutorial_/index.html#defining-abi-stable-types
- [6] M. O. Manero, "Plugins in Rust: Getting our Hands Dirty." Accessed: Apr. 02, 2026. [Online]. Available: <https://nullderef.com/blog/plugin-impl/>
- [7] "Crate criterion." Accessed: Apr. 06, 2026. [Online]. Available: <https://docs.rs/criterion/0.8.2/criterion/>
- [8] gungraun developers, "Crate gungraun." Accessed: Apr. 06, 2026. [Online]. Available: <https://docs.rs/gungraun/0.17.2/gungraun/>
- [9] T. R. Foundation, "Function black_box." Accessed: Apr. 06, 2026. [Online]. Available: https://doc.rust-lang.org/beta/std/hint/fn.black_box.html
- [10] P. LeVasseur, "What does it take to ship Rust in safety-critical?." Accessed: Apr. 14, 2026. [Online]. Available: <https://blog.rust-lang.org/2026/01/14/what-does-it-take-to-ship-rust-in-safety-critical/>
- [11] Ferrocene, "Ferrous Systems Achieves IEC 61508 (SIL 2) Certification for Rust Core Library Subset." Accessed: Apr. 14, 2026. [Online]. Available: <https://ferrous-systems.com/blog/ferrocene-libcore-news-release/>